

“板凳龙”运动机理模型探究

摘要

板凳龙舞是中国非物质文化遗产的一部分，拥有千年历史。其舞蹈利用连成一串的板凳模拟龙的形态，通过沿螺旋路径移动展现龙的活力。本文基于对“**板凳龙**”运动机理模型的研究，旨在优化表演，提高安全性，还有助于深入了解和传承这种独特的文化艺术形式。

针对问题一，通过建立**逆向螺旋线模型**，描述“板凳龙”在表演过程中的运动轨迹与速度变化。首先，基于极坐标方程表示螺旋线的逆向运动，并将其转换为直角坐标，方便计算舞龙过程中每节板凳的具体位置。接着，通过龙头、龙身和龙尾的运动状态建立“**板凳龙**”运动机理模型，结合**滞后效应**，推导出各部分的切向速度和角速度。最后，通过递推公式逐步计算各板凳的位置与速度。

针对问题二，在问题一的基础上增加了**碰撞检测**的条件，分析在盘入过程中板凳之间可能发生的物理碰撞。首先考虑板凳之间的插接式设计和角度变化，建立角度变化公式；然后确立碰撞检测条件，分别分析长度和宽度方向的碰撞可能性；利用**轴对齐边界框（AABB）思想**进行宽度方向的碰撞检测，并结合插接式设计的角度变化影响，调整有效宽度距离；最后通过模型求解确定盘入的终止时刻为 $412.583s$ ，确保在该时刻前板凳之间不会发生碰撞。

针对问题三，通过建立“**板凳龙**”**最小螺距优化模型**，求解在调头空间有限的情况下，从顺时针盘入到逆时针盘出时的最小螺距。首先设定螺距为决策变量，并以最小化螺线总长度建立目标函数。结合掉头空间的物理和几何约束，确保龙头顺利进入并完成掉头。优化过程采用**梯度下降法**，迭代计算目标函数的梯度，逐步更新螺距值，最终得到满足所有约束条件的最小螺距为 $45.035cm$ 。

针对问题四，通过建立几何模型，优化掉头时螺旋线与圆弧的平滑连接，确保盘入和盘出过程中螺旋线的自然过渡。首先构建掉头路径的几何模型，定义前后两段圆弧的半径，并根据对称螺旋线的原则，确保盘入和盘出螺旋线的连续性。接着将龙头的极坐标转换为笛卡尔坐标，计算螺旋线与圆弧的切点，确保切线方向一致，实现光滑过渡。建立“**板凳龙**”**路径规划几何模型**，分析其运动过程中速度和滞后效应。通过简单路径搜索法，得到优化总路径长度一致，前后圆弧的半径分别为 $R_1 = 2,902, R_2 = 1.451m$ ，总路径长度最小。

针对问题五，本文首先建立“**板凳龙**”**节点运动特征模型**，考虑龙头固定线速度下，通过不同曲率半径时的角速度变化。确立了龙尾作为最小曲率半径节点，依据此模型，计算各节点速度，并根据速度上限约束条件，应用**二分法**逐步调整龙头速度，以确保全部节点的速度均符合要求，最终找到龙头的最大可行速度为 $1.538m/s$ 。

关键词：逆向螺旋线模型 滞后效应 轴对齐边界框（AABB）思想 二分法

一、问题重述

板凳龙是源于中国古代农耕文化的传统民间舞蹈，已有千年历史，是非物质文化遗产的重要组成部分。表演道具由多条板凳连接成龙形，龙头较长，龙身和龙尾由较短板凳相连，装饰有五彩龙鳞。



图 1 传统板凳龙舞的螺旋形表演场景

表演者通过手持把手，以螺线为行进路线，进行运动，展现龙的形象和流畅的舞蹈动作（如图 1）。通过分析板凳龙的速度、长度及行进路线，可优化表演方案，避免碰撞，提升表演效果，探究其中蕴含的传统文化机理。

问题要求：

板凳龙由龙头、龙身、龙尾三部分组成，总共使用 223 条板凳。具体分配如下：龙头：由 1 条板凳构成，板凳长度为 341cm，宽度为 30cm。龙身：由 221 条长度为 220cm、宽度为 30cm 的板凳组成。龙尾：由 1 条板凳构成，板凳长度为 220cm，宽度为 30cm。每条板凳上有两个圆孔，孔径为 5.5cm，圆孔中心距离板凳最窄边的距离为 27.5cm。

现需解决如下问题：

问题一：若板凳龙的行进路线为等距螺线，螺距和盘入起点已知，龙头速度恒定。在假定各龙把手均在螺线上的情况下，0s 至 300s 内每秒各个龙把手的位置、速度。

问题二：若继续按照第一问设定的等距螺线盘入，在其他条件相同的情况下，探究舞龙队在行进过程中出现碰撞的时刻。

问题三：要求在等距螺线共原点的基础上，给定一个半径为 4.5m 的圆形区域，作为掉头盘出的可操作空间，要求确定其最小螺距，使得板凳龙的龙头前把手能够沿着螺线恰好盘入可操作空间的边缘。

问题四：给定螺线螺距为 1.7m，同时盘入螺线与盘出螺线关于螺线中心对称，其中前半圆弧半径是后半圆弧的两倍，并与盘入螺线，盘出螺线分别相切。在不影响题目要求的情况下，设法缩短掉头曲线。并求以掉头开始为零时刻，计算-100s 至 100s 内每秒舞龙队的位置和速度。

问题五：若在问题四求解的螺线路径上行进，计算出龙头的最大速度，使得龙头保持在该速度下前进时各龙把手的速度均不超过 2m/s。

二、问题分析

2.1 问题一分析

问题一通过构建螺旋线运动模型，首先，基于极坐标系的螺旋线方程，描述龙头沿螺旋线移动的路径，并结合龙头切向速度恒定的特点，建立了螺旋线的逆向运动模型。在此基础上，将螺旋线的极坐标转换为直角坐标，以便计算每节板凳把手的位置接下来，分别对龙头、龙身和龙尾的运动特性进行了建模分析，龙头的切向速度保持恒定，而龙身和龙尾的速度则由于滞后效应在不同时间点发生变化。通过分析各节板凳的滞后时间和相对位置，得出了每节板凳的速度分解式，具体计算了切向速度和角速度的变化。最后，利用递推方法分步计算每一秒钟内龙头、龙身和龙尾的前把手位置和速度，结合相邻时刻的位移变化求出速度值。该方法通过设定时间步长（每秒一帧），逐帧计算各节板凳的运动状态。

2.2 问题二分析

问题二在问题一的基础上，增加了碰撞检测条件，分析“板凳龙”盘入螺旋线时，板凳之间可能发生的物理碰撞，求解舞龙队无法继续盘入的时刻。首先考虑板凳之间的插接式设计和角度变化，建立螺旋线角度变化模型。然后确立碰撞检测条件，分别从长度和宽度方向进行分析。长度方向上，当相邻板凳的前把手距离小于其固定长度时会发生碰撞；宽度方向上，利用轴对齐边界框（AABB）思想^[1]，当相邻螺旋线间的径向距离小于板凳的宽度时会发生碰撞。最后结合插接式设计影响的角度变化，建立综合碰撞检测模型，递推计算得出舞龙队无法继续盘入的终止时刻。

2.3 问题三分析

问题三求解在有限调头空间内顺时针盘入并逆时针盘出时的最小螺距，使龙头能顺利完成调头并满足所有物理和几何约束。首先将螺距设为决策变量，模型的目标是找到最小的螺距值，从而最小化螺旋线的总长度。在此过程中，须满足两个关键几何约束条件：一是调头空间的半径约束，确保龙头能够盘入调头空间且不超出其边界；二是宽度方向上的碰撞检测约束，防止相邻螺旋线上的板凳在宽度方向上发生物理碰撞。结合约束条件，模型确保舞龙队的运动轨迹符合实际物理要求。接着通过梯度下降法来优化问题。首先定义目标函数，以螺距最小化为目标，并设定初始螺距值。通过计算目标函数的梯度，逐步更新螺距参数。每次迭代中，根据初始螺距值和约束条件，计算路径的总长度，并通过求导得到梯度。利用梯度下降法对螺距进行更新，逐步缩小误差，直到螺距收敛到最优值。

2.4 问题四分析

问题四要求在掉头时，螺旋线与掉头路径中的圆弧能够自然、平滑地连接，同时满足物理和几何约束。首先建立掉头路径的几何模型，前后两段圆弧的半径满足前段是后段的两倍，根据螺旋线的对称性确保盘入和盘出路径连续。通过极坐标与笛卡尔坐标的转换，准确描述龙头的运动轨迹，并结合圆弧方程，确保路径的平滑过渡。随后建立龙身和龙尾的运动模型，考虑时间滞后效应来计算它们的运动和速度。通过计算螺旋线与圆弧的相切点，确保两者切线方向一致。最后利用简单路径搜索法，调整圆弧半径，找到总路径长度最小的最优方案。

2.5 问题五分析

在问题五中，可首先确立了舞龙队各节点的运动特征。由于龙头的线速度是固定的可以分析龙头在不同曲线半径处的角速度变化。这一变化反映在龙头在较大曲线半径处的角速度较小，而在较小曲线半径处的角速度较大。接着通过调头路径的设计，确定龙尾处于最小曲率半径的节点，从而计算出龙尾的速度。可将舞龙队的速度上限作为约束条件，确保所有速度节点均满足此。最后采用二分法通过逐步缩小搜索区间，每次迭代计算区间的中间值，并检查该速度是否使得所有节点的速度满足题目要求。如果当前速度符合要求，则尝试增大速度以探索更高的可能值；如果速度过高导致某些节点超速，则降低速度。这个迭代过程持续进行，直到找到最接近的最大允许速度，以确保所有节点速度都在安全和规定的范围之内。

三、模型假设

1. 假设板凳间精确连接，忽略连接误差或舞动过程误差。
2. 假设舞龙过程为二维平面问题，忽略地形起伏等三维因素。
3. 假设只考虑相邻板凳在长度和宽度方向的碰撞，不考虑其他碰撞形式。
4. 假设在模型中，舞龙过程不受空气阻力和摩擦力的影响。
5. 假设掉头过程中，螺旋线与两段圆弧平滑连接，满足相切条件。

四、符号说明

符号	说明	单位
m_d	等距螺旋线的螺距	m
Δt_n	第 n 阶板凳与龙头的滞后时间	s
$d_{n \rightarrow n+1}(t)$	相邻两节板凳的前把手之间的距离	m
$d_i(t)$	相邻螺旋线上的板凳在径向方向的距离	m
d_η	板凳固定宽度	m
d_φ	插接式设计影响下的实际宽度距离	m
$P_3(x_3, y_3)$	盘出螺旋线与后段圆弧的相切点	
r_{begin}	掉头空间内的起始径向距离	m
$L_{(m_d)}$	最小螺旋线的总长度	m
ν_i	第 i 个节点的线速度大小	rad/s

五、模型的建立与求解

5.1 问题一模型的建立与求解

问题一主要通过建立逆向螺旋线运动模型，描述“板凳龙”在表演过程中的位置和速度变化，并基于此模型求解舞龙队伍的运动状态。

5.1.1 逆向螺旋线运动模型的建立

根据假设 1，舞龙过程中的螺旋线为理想化的二维路径。螺旋线的数学表示在极坐标系下，其标准方程^[2]可以定义为：

$$r = a + b\theta \quad (1)$$

其中， r 表示径向距离， a 表示螺旋线的初始位置， θ 表示旋转的角度， b 表示螺旋线的螺距。

根据题意可知，螺旋线的半径 r 随角度 θ 的增大而线性减小，且每旋转一周 (2π 或 360°)，半径减小 $0.55m$ 。由此可得，龙头沿螺旋线移动的逆向螺旋线方程可表示为：

$$r(\theta) = r_0 - \frac{m_d}{2\pi} \theta \quad (2)$$

其中， $r_0 = 8.8m$ 表示从螺旋线上第 16 圈开始的初始位置。

对于螺旋线上的每个点，其位置随时间的推移持续受两个因素的影响：半径 $r(t)$ 随时间的减小和角度 $\theta(t)$ 随时间的变化。

5.1.2 极坐标与直角坐标的转化

根据题目条件中的“舞龙队沿螺距为 $55cm$ 的等距螺线顺时针盘入，各把手中心均位于螺线上”即不需要考虑各节板凳之间的轴承连接带来的系统性误差，各个把手中心的位置可以从极坐标 (r, θ) 转变为二维直角坐标 (x, y) ，其中：

$$\begin{aligned} x(t) &= r(t) \cos[\theta(t)] \\ y(t) &= r(t) \sin[\theta(t)] \end{aligned} \quad (3)$$

其中， $r(t)$ 表示螺旋线在时间 t 时所对应的半径， $\theta(t)$ 表示螺旋线在时间 t 时所对应的角度。

5.1.3 基于“板凳龙”运动机理模型的建立

由已知条件，可得龙头前把手的速度始终保持 $1m/s$ ，但由于舞龙队沿着曲线运动，其他板凳的速度会随之不断改变。对于第 n 节板凳，其速度应分为切向速度和角速度（法向速度）两个部分：

1. 龙头运动建模过程

龙头的切向速度恒定 $v = 1m/s$ 。由于切向速度与角速度之间的关系式为：

$$v = r(t)\omega(t) \quad (4)$$

其中， $\omega(t)$ 是时间为 t 时的角速度，表示旋转角度 $\theta(t)$ 对时间的变化率，即：

$$\omega(t) = \frac{d\theta(t)}{dt} \quad (5)$$

根据物理公式又可得：

$$\omega(t) = \frac{v}{r(t)} = \frac{1}{r(t)} \quad (6)$$

因为螺旋线的螺距 $m_d = 0.55m$ ，在舞龙队盘入过程中，当龙头沿螺旋线方向前进时，每旋转一圈（即角度增加 2π 弧度）时，其沿径向方向的距离会减小 m_d ，结合式(4) (5)，可得在队伍盘入过程中，每经过一段时间， $\theta(t)$ 会以恒定的速率增加。

由于龙头的切向速度恒定，根据上文内容可知 $\theta(t)$ 可通过螺距 m_d 和盘入过程中龙头的速度变化决定。其中螺旋线以切向速度旋转一圈所需要的时间为 $t = m_d/v$ ，已知物体旋转一圈的时间 t ，则与之对应的角速度为：

$$\omega(t) = \frac{\Delta\theta}{\Delta t} = \frac{2\pi}{t} = \frac{2\pi v}{m_d} = \frac{2\pi}{m_d} \quad (7)$$

即可得旋转角度 $\theta(t)$ 随时间的变化关系为：

$$\theta(t) = \int_0^t \omega(t') dt' = \frac{2\pi}{m_d} t \quad (8)$$

由式(8)得，旋转角度 $\theta(t)$ 随时间呈线性增加，且增加的速率由螺距 m_d 决定。因此，龙头在时间 t 上的位置关系可表示为：

$$\begin{aligned} x(t) &= r(t) \cos \left[\frac{2\pi}{m_d} t \right] \\ y(t) &= r(t) \sin \left[\frac{2\pi}{m_d} t \right] \end{aligned} \quad (9)$$

其中

$$r(t) = r_0 - \frac{m_d}{2\pi} \theta(t) \quad (10)$$

2.龙身、龙尾运动建模过程

(1) 龙身龙尾的滞后效应

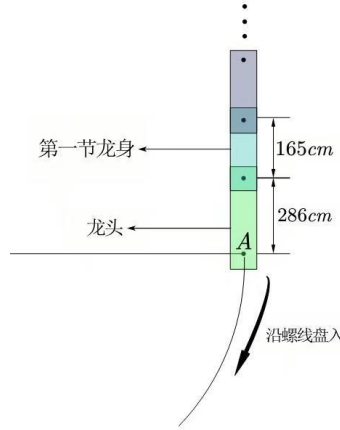


图 2 滞后效应效果呈现图

如图所示：龙头与第一节龙身之间前后把手之间的距离为 $L_1 = 286\text{cm}$ ，龙身与龙身每节板凳之间前后把手之间的距离为 $L_2 = 165\text{cm}$ 。那么第 n 阶板凳相对于龙头的滞后时间为：

$$\Delta t_n = L_1 + L_2(n - 1) \quad (11)$$

(2) 龙身龙尾的位置计算

综上，每节龙身的前把手的位置可以通过时间滞后公式进行计算：

$$\begin{aligned} x_n(t) &= r(t - \Delta t_n) \cos[\theta(t - \Delta t_n)] \\ y_n(t) &= r(t - \Delta t_n) \sin[\theta(t - \Delta t_n)] \end{aligned} \quad (12)$$

(3) 龙身龙尾的位置计算

在盘入过程中，龙身龙尾的速度可以通过其相应位置对时间的变化率进行计算，对每个板凳前把手的 $x_n(t)$ 和 $y_n(t)$ 求导，可得对应的速度分量为：

$$\begin{aligned} v_{x_n(t)} &= \frac{dx_n(t)}{dt} \\ v_{y_n(t)} &= \frac{dy_n(t)}{dt} \end{aligned} \quad (13)$$

5.1.4 问题一舞龙队位置和速度的求解

通过建立的“板凳龙”运动模型，具体求解过程如下：

位置和速度求解：

Step 1: 时间步长设置

将总时间 300 秒分为每秒一帧，每帧对应的时间间隔为 1 秒。

Step 2: 位置递推计算

1. 根据螺旋线方程计算龙头在每一时刻的位置。
2. 使用递推公式，依次计算龙身和龙尾的各节板凳在每一帧的位置信息。

Step 3: 速度计算

1. 通过相邻时刻每节板凳位置的变化，计算其速度。
2. 速度等于相邻帧之间位置变化除以时间步长。

Step 4: 数据记录及结果输出

将每秒龙头、龙身及龙尾的前把手位置和速度数据及特定时刻数据。

由以上过程利用 python 进行求解，得到每秒整个舞龙队的位置和速度，详细结果见附件 result1.xls，特定时刻的求解结果见下表 1，表 2：

表 1 问题一龙头龙尾龙身位置

	0s	60s	120s	180s	240s	300s
龙头 $x(m)$	8.800000	5.79112	-4.085032	-2.963334	-2.594165	4.420479
龙头 $y(m)$	0.000000	-5.711327	-6.305411	6.094903	-5.356889	2.319891
第 1 节 $x(m)$	8.363801	7.456665	-1.445657	-5.236954	4.820640	2.459913
第 1 节 $y(m)$	2.826543	-3.440465	-7.405749	4.359823	-3.572145	4.402232
第 51 节 $x(m)$	-9.518729	-8.686304	-5.543155	2.890481	5.979894	-6.301271
第 51 节 $y(m)$	1.340899	2.539901	6.377854	7.249235	-3.827851	0.466075
第 101 节 $x(m)$	2.914342	5.697475	5.362181	1.899022	-4.917235	-6.237689
第 101 节 $y(m)$	-9.719021	-8.001103	-7.557384	-8.471650	-6.379930	3.935980
第 151 节 $x(m)$	10.86204	6.691845	2.388214	10.04678	2.964967	7.040578
第 151 节 $y(m)$	1.829220	8.134695	9.727741	9.424789	8.399827	4.393231
第 201 节 $x(m)$	4.554582	6.620214	-10.62751	-9.287413	-7.456875	-7.458381
第 201 节 $y(m)$	10.72511	9.025143	1.358964	-4.247263	-6.181224	-5.263828
龙尾 (后) $x(m)$	-5.304657	7.376510	10.97401	7.383357	3.240302	1.784406
龙尾 (后) $y(m)$	-10.67742	-8.797451	8.441890	7.412871	9.469542	9.301308

表 2 问题一龙头龙尾龙身速度

	0s	60s	120s	180s	240s	300s
龙头 (m/s)	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节 (m/s)	0.999970	0.999960	0.999944	0.999912	0.999852	0.999706
第 51 节 (m/s)	0.999751	0.999739	0.999334	0.999375	0.999058	0.998651
第 101 节 (m/s)	0.998894	0.999090	0.999190	0.998972	0.998334	0.998070
第 151 节 (m/s)	0.998650	0.998816	0.998916	0.998406	0.997770	0.997826
第 201 节 (m/s)	0.997827	0.998405	0.998695	0.997126	0.998712	0.997280
龙尾后 (m/s)	0.997781	0.997916	0.998201	0.997292	0.997213	0.997344

5.2 问题二模型的建立与求解

问题二在问题一的基础上，添加了碰撞检测这一条件，求舞龙队在沿螺距为 55cm 的螺线盘入过程中，不能继续盘入的时刻。这是由于板凳之间的碰撞所致，综合假设 3，故只需考虑螺旋线的几何特性、板凳的插接式设计、宽度和长度的影响等三个方面的因素，为此本题大致的求解思路如下：

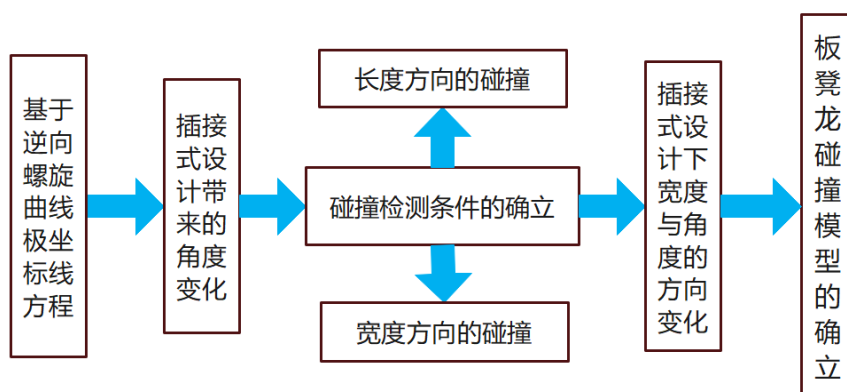


图 3 问题二建模流程图

5.2.1 插接式设计的角度变化

如图 4，由于板凳之间的连接方式为插接式，相邻两节之间有凹凸槽设计，可以插接到一起，形成连续的龙身，因而在各个板凳之间会形成一定的夹角，此过程中的角度变化会诱发碰撞这一状态。

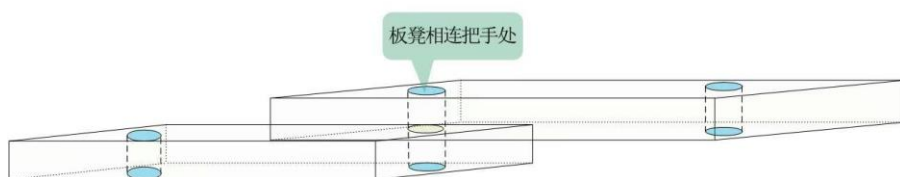


图 4 插接式设计示意图

在“板凳龙”盘入的过程中，其间发生的角度变化，可以通过螺旋线的角度进而确定，即：

$$\theta_{n+1}(t) - \theta_n(t) = \frac{2\pi}{m_d} (t_{n+1} - t_n) \quad (14)$$

其中， $\theta_n(t)$ 和 $\theta_{n+1}(t)$ 是相邻两节板凳的角度变化。

5.2.2 碰撞检测条件的确立

通过 python 绘制出的预测临界示意图，可化抽象为具体，考虑螺旋线的几何特性时，可进一步细分为长度的影响、宽度的影响和板凳的插接式设计等三个方面的因素。以下分别对这三种影响因素进行研究。

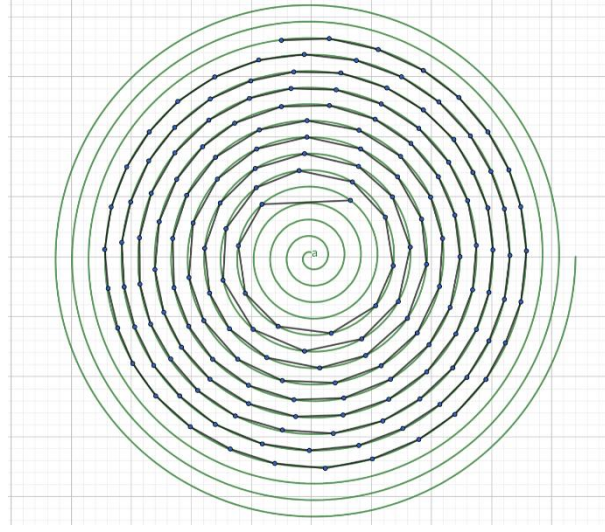


图 5 预测碰撞示意图

1.长度方向碰撞条件

通过分析题目条件可得，相邻两节板凳的中心位置距离不应该小于其固定长度 L_1 或 L_2 ，相邻两节板凳的前把手之间的距离 $d_{n \rightarrow n+1}(t)$ 为:

$$d_{n \rightarrow n+1}(t) = \sqrt{[x_{n+1}(t) - x_n(t)]^2 + [y_{n+1}(t) - y_n(t)]^2} \quad (15)$$

其中，当满足 $d_{n \rightarrow n+1}(t) \leq L_1(L_2)$ 时，其长度方面发生物理碰撞。

2.宽度方向碰撞条件

当板凳位于螺旋线的不同半径之间时（如下图所示），会因为相邻螺旋线宽度方向上的板凳之间的距离过小而发生碰撞，对此可采用包围盒检测法^[1]中的轴对齐边界框（AABB）检测思想进行检测。

当相邻螺旋线上的板凳在径向方向的距离 $d_i(t)$ 小于板凳固定宽度 d_η 时，发生碰撞，即碰撞的条件为:

$$d_i(t) = |r_n(t) - r_{n+1}(t)| \quad (16)$$

其中，当 $d_i(t) \leq d_\eta$ 时，其宽度方面发生物理碰撞。

3.插接式设计下的宽度与角度方向

由于板凳之间为插接式连接，各个板凳间存在一定的夹角变化，从而带来宽度方向上的碰撞检测，故需考虑相邻板凳之间的角度变化 $\Delta\theta$ ：

$$\Delta\theta = \theta_{n+1}(t) - \theta_n(t) \quad (17)$$

相邻板凳之间在宽度方向上的有效距离需根据实际角度变化调整，在插接式设计影响下的实际宽度距离 d_φ 为:

$$d_{\varphi}(t) = d_i(t) - \Delta\theta d_{\eta} \quad (18)$$

5.2.3 碰撞检测模型的确立

由于本文模型中假设舞龙队在盘入的过程中，各把手中心均位于螺线上。结合长度和宽度方向的条件，可知在长度方向，相邻两节板凳的中心位置间的距离恒等于其固定长度 L_1 或 L_2 。

故在本文中长度方向的碰撞可忽略不计，则最终的碰撞检测公式为结合插接式设计宽度方向的碰撞：

$$\begin{aligned} d_{\varphi}(t) &= d_i(t) - \Delta\theta d_{\eta} \\ d_{\varphi}(t) &\leq d_{\eta} \end{aligned} \quad (19)$$

5.2.4 模型二终止时刻的求解

与问题一类似，通过设置相同的时间步长，并利用 Python 进行求解。在此基础上，逐步计算舞龙队在盘入过程中每个时间步的运动状态，最终得出舞龙队盘入的终止时刻为412.583s。此时，确保板凳之间不会发生碰撞，且符合所有物理和几何约束条件。计算结果包括舞龙队在各个时刻的位置和速度，详细数据存储在附件 result2.xls 中，特定条件下的求解结果汇总于下表 3：

表 3 问题二终止时刻下特定条件的位置及速度

	横坐标 $x(m)$	纵坐标 $y(m)$	速度 $v(m/s)$
龙头	2.010258	2.724506	0.999685
第 1 节	-0.723539	3.368075	0.999194
第 51 节	-3.540740	3.771070	0.998934
第 101 节	-3.179092	-5.576498	0.997146
第 151 节	-6.512650	-3.650617	0.992337
第 201 节	-0.377405	8.386457	0.989695
龙尾后	8.125781	-3.300179	0.987191

5.3 问题三模型的建立与求解

问题三在调头空间一定的条件下，舞龙队将由顺时针盘入调头切换为逆时针盘出时，求解最小螺距，使得舞龙队的龙头能顺利盘入调头空间，并且满足相关的物理和几何约束。由此，本文根据上述约束条件建立目标优化模型^[3]进行求解。

5.3.1 “板凳龙”最小螺距优化模型的确立

1.决策变量

问题三的核心目标为求解最小螺距 m'_d ，其目标方程可表示为：

$$\min m'_d \quad (20)$$

2.约束条件

除了最小螺距 m'_d 的目标外，还需要满足以下关键约束条件：

(1) 掉头空间内的约束

为确保龙头顺利进入调头空间内，螺旋线的最小半径 $r_{\min}$ 需满足

$$r_{\min} \geq r_1 \quad (21)$$

其中 $r_1 = 4.5m$ ，表示掉头空间的半径大小。结合式 (2)，则进入掉头空间内的边界条件为：

$$r_2 = r_{begin} - \frac{m'_d}{2\pi} \theta \quad (22)$$

其中， r_{begin} 表示掉头空间内的起始径向距离。

为确保龙头在盘入的过程中不会超出掉头空间的边界，则此时进入掉头空间内的边界条件为：

$$r_{\min} = r_{begin} - \frac{m'_d}{2\pi} \theta_{\min} \geq r_1 \quad (23)$$

则此时龙头前把手在新螺旋线方程条件下的位置为：

$$\begin{aligned} x_{\varphi}(t) &= r_2(\theta) \cos[\theta(t)] = \left[r_{begin} - \frac{m'_d}{2\pi} \theta(t) \right] \cos\left(\frac{m'_d}{2\pi} t\right) \\ y_{\varphi}(t) &= r_2(\theta) \sin[\theta(t)] = \left[r_{begin} - \frac{m'_d}{2\pi} \theta(t) \right] \sin\left(\frac{m'_d}{2\pi} t\right) \end{aligned} \quad (24)$$

(2) 插接式设计下宽度方向碰撞检测约束

由问题二模型建立过程分析可知，在盘入过程中，相邻螺旋线上的板凳在宽度方向上发生碰撞需满足：

$$\begin{aligned} d_i(t) &= |r_{2n}(t) - r_{2n+1}(t)| \geq d_{\eta} \\ d_{\varphi}(t) &= d_i(t) - \Delta\theta d_{\eta} \geq d_{\eta} \end{aligned} \quad (25)$$

综上可得，最小螺距优化模型为：

$$\min m'_d \quad (26)$$

$$s.t. \begin{cases} r_{\min} = r_{begin} - \frac{m'_d}{2\pi} \theta_{\min} \geq r_1 \\ d_{\varphi \rightarrow \varphi+1} = \sqrt{[x_{\varphi+1}(t) - x_{\varphi}(t)]^2 + [y_{\varphi+1}(t) - y_{\varphi}(t)]^2} \\ d_i(t) = |r_{2n}(t) - r_{2n+1}(t)| \geq d_{\eta} \\ d_{\varphi}(t) = d_i(t) - \Delta\theta d_{\eta} \geq d_{\eta} \end{cases} \quad (27)$$

5.3.2 基于梯度下降法^[4]求解最小螺距优化模型

基于梯度下降法的最小螺距优化模型，可以灵活应用于不同的模型和约束条件，帮助求解实际问题中的复杂优化目标。具体求解步骤如下：

Step1 目标函数的定义：

最小螺距优化模型的总目标为最小螺线的总长度最小，设总长度为 $L_{(m_{dl})}$ ，其中 m_{dl} 是螺距，则有：

$$L_{(m_{dl})} = 2\pi n m_{dl} \quad (28)$$

其核心目标为找到最小的螺距 m_{dl} ，使得龙头能够到达调头区域，即总长度为 $L_{(m_{dl})}$ 最小。

Step2 梯度优化计算：

1.初始化螺距 m_{dl}

首先，需要选择一个初始的螺距 $m_{dl} = 1.50cm$ 。

2.目标函数 $L_{(m_{dl})}$ 的梯度计算

在螺旋运动中，总路径长度 $L_{(m_{dl})}$ 可近似为式(28),通过对 m_{dl} 求导得

$$\frac{\partial L_{(m_{dl})}}{\partial m_{dl}} = -\frac{C}{m_{dl}^2} \quad (29)$$

3.目标函数 $L_{(m_{dl})}$ 的梯度更新

使用梯度下降法进行参数更新：

$$m_{dl1} = m_{dl2} - \phi \frac{\partial L_{(m_{dl})}}{\partial m_{dl}} \quad (30)$$

其中， m_{dl1} 表示更新后的螺距， m_{dl2} 表示上一次迭代的螺距， $\phi = 0.01$ 表示学习步长，决定每次更新的幅度。

4.目标函数 $L_{(m_{dl})}$ 的迭代收敛

重复上述更新步骤，直到梯度足够小，或者达到最大迭代次数。

由以上过程，得到最小螺距为 $45.035cm$ 时，龙头前把手能够沿着相应的螺线盘入到调头空间的边界。

5.4 问题四模型的建立与求解

根据假设 5，在建模过程中能够确保舞龙队掉头时的螺旋线与盘入时的螺旋线能够自然连接。掉头路径的各部分（盘入螺线、前段圆弧、后段圆弧、盘出螺线）之间保持光滑衔接。在设计掉头路径时，需精确调整前后两段圆弧的半径及连接点的位置，以满足这些优化条件，从而实现理想的掉头效果。

5.4.1 “板凳龙” 路径规划几何模型的建立

掉头路径如下图所示，由前段和后段两段圆弧组成，假设前端圆弧的半径为 R_1 ，后段圆弧的半径为 R_2 ，根据题目中指出“前一段圆弧的半径是后一段的2倍”，有 $R_1 = 2R_2$ 满足。

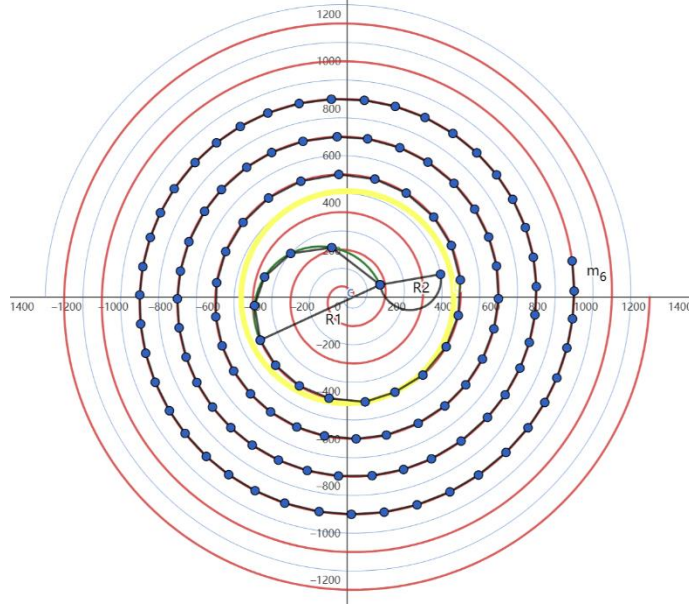


图 6 掉头路径示意图

由于盘出螺线与盘入螺线关于螺线中心呈中心对称，即盘出是对盘入的对称螺旋线。而在本题中（如图 6）盘入的轨迹明确，且旋转方向、螺距和初始位置已知，故舞龙队盘入时的螺旋线轨迹可视为已知。

盘入螺旋线的初始位置不涉及固定圈数的概念，螺旋线方程从时间 $t=0$ 开始计算的，时间可视为 $t=0$ 开始，故有：

$$r_{\beta} = \alpha \theta_{\beta} \quad (31)$$

其中， $\alpha = 0.2708$ 是螺距常数。

5.4.2 龙头极坐标与笛卡尔坐标的转换

易知，舞龙队龙头的两坐标系下位置互换为：

$$\begin{aligned} x_{\beta}(\theta) &= r_{\beta}(\theta_{\beta}) \cos(\theta_{\beta}) \\ y_{\beta}(\theta) &= r_{\beta}(\theta_{\beta}) \sin(\theta_{\beta}) \end{aligned} \quad (32)$$

根据问题一，同理可得时间与角度 θ_{β} 的关系仍为线性关系，且随着时间 t 的推移，角度 θ_{β} 线性增加。

设前段圆弧的半径为 R_1 ，圆心坐标为 $O(x_0, y_0)$ ，前半段圆弧的方程可表示为：

$$(x - x_{01})^2 + (y - y_{01})^2 = R_1^2 \quad (33)$$

同理，后半段圆弧的方程可表示为：

$$(x - x_{02})^2 + (y - y_{02})^2 = R_2^2 \quad (34)$$

5.4.3 龙尾龙身运动模型的建立

同理，由于时间的滞后效应，龙尾龙身的位置公式可表示为：

$$\begin{aligned} x_\alpha(t) &= r_\beta(\theta_\beta - \Delta t) \cos(\theta_\beta - \Delta t) \\ y_\alpha(t) &= r_\beta(\theta_\beta - \Delta t) \sin(\theta_\beta - \Delta t) \end{aligned} \quad (35)$$

滞后时间为：

$$\Delta t = L_1 + L_2(n - 1) \quad (36)$$

进一步分别进行求导可得龙身、龙尾的速度以及其切向合速度 v_α ：

$$\begin{aligned} v_{x_\alpha(t)} &= \frac{dx_\alpha(t)}{dt} \\ v_{y_\alpha(t)} &= \frac{dy_\alpha(t)}{dt} \\ v_\alpha &= \sqrt{v_{x_\alpha(t)}^2 + v_{y_\alpha(t)}^2} \end{aligned} \quad (37)$$

5.4.4 螺旋线与圆弧相切点的确定

在问题四中，通过长度较大的龙头带动龙身和龙尾的运动，故而龙头的临界条件及相切，可以保证整个舞龙队的运动畅通无阻。螺旋线与圆弧须具有相同的切线方向，在极坐标系中，螺旋线的切线方向可表示为：

$$\frac{dy_\beta}{dx_\beta} = \frac{r'_\beta(\theta_\beta) \sin \theta_\beta + r_\beta(\theta_\beta) \cos \theta_\beta}{r'_\beta(\theta_\beta) \cos \theta_\beta - r_\beta(\theta_\beta) \sin \theta_\beta} \quad (38)$$

圆弧的切线斜率为：

$$\frac{dy}{dx} = -\frac{x}{y} \quad (39)$$

螺旋线与圆弧具有相同的切线方向，因此其斜率大小必相等，进而可求解出切点 $P_1(x_1, y_1)$ 和对应的角度 θ_1 。又因为两者满足在相切点 $P_2(x_2, y_2)$ 处的斜率一致，又有：

$$\frac{y_2 - y_{01}}{x_2 - x_{01}} = \frac{y_2 - y_{02}}{x_2 - x_{02}} \quad (40)$$

通过式(40)，可确定前后段圆弧的切点 $P_2(x_2, y_2)$ 。同理可求解出盘出螺旋线与后段圆弧的相切点 $P_3(x_3, y_3)$ 和对应的角度 θ_3 。

5. 4. 5 简单路径搜索法优化路径

简单的数值搜索方法是一种实用的优化手段。在本题的实际背景下，这种方法不需要复杂的数学推导，只需要通过数值计算尝试不同的半径 R_1, R_2 ，并找到使总路径长度最小的值。其中总路径长度为：两段圆弧的总长度之和。

通过设定初始值，并逐步调整，最终求得最优解为 $R_1 = 2,902, R_2 = 1.451m$ 时，仍保持各部分相切，使得总路径长度最小。

由以上过程利用 python 进行求解，得到每秒整个舞龙队的位置和速度，详细结果见附件 result4.xls，特定时刻的求解结果见下表 4，表 5：

表 4 问题四调头后特定条件的速度

	- 100s	- 50s	0s	50s	100s
龙头 x (m)	-2.250561	1.79112	-1.451032	-2.813334	-6.156665
龙头 y (m)	7.905154	6.711327	-0.355411	5.964903	-5.700889
第 1 节 x (m)	-4.963801	-1.456665	-0.000657	-0.000954	3.820640
第 1 节 y (m)	6.726543	-6.440465	-2.485749	6.459823	-7.372145
第 51 节 x (m)	-1.341729	-8.686304	7.703155	3.210481	2.579894
第 51 节 y (m)	-10.60632	3.539901	0.67754	-4.812425	-4.182781
第 101 节 x (m)	7.896255	3.659815	1.0.08612	-5.559022	-4.287235
第 101 节 y (m)	-9.832511	10.88621	1.867384	-6.871650	-5.779930
第 151 节 x (m)	7.272582	-0.726770	-7.852962	10.07678	3.044967
第 151 节 y (m)	-12.27731	13.27613	-9.384511	-2.691654	9.301554
第 201 节 x (m)	-3.945112	5.133160	-8.68253	12.19106	10.15247
第 201 节 y (m)	-15.72511	13.025143	10.90364	4.346815	6.114905
龙尾（后） x (m)	114.82045	-13.95131	13.9856	-13.65892	9.512614
龙尾（后） y (m)	6.949235	-6.758115	4.299477	0.972175	-8.345123

表 5 问题四调头后特定条件的速度

	- 100s	- 50s	0s	50s	100s
龙头(m/s)	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节(m/s)	0.999959	0.999641	1.009994	1.00012	1.009852
第 51 节(m/s)	0.999740	0.998834	0.986334	1.030033	1.000194
第 101 节(m/s)	0.999092	0.997983	0.985180	1.029872	0.918334
第 151 节(m/s)	0.998650	0.997837	0.985337	1.025406	0.917770
第 201 节(m/s)	0.998405	0.997749	0.983695	1.025126	0.918712
龙尾后(m/s)	0.997781	0.997791	0.981191	1.0236957	0.917213

5. 5 模型五的建立与求解

问题五需根据调头路径的几何特征求解，确定舞龙队在不同的位置时的速度，并通过调整龙头的速度来确保所有节点的速度都在给定的范围之内。

5. 5. 1 舞龙队各节点运动特征的确立

由于龙头行进速度保持不变，设其线速度大小为 ν_0 。在通过调头路径的设计后，需

考虑龙头所处曲线半径的角速度变化。当龙头处于较大曲线半径时，沿着较长轨迹运动，龙头的角速度较小；当龙头处于较小曲线半径时，沿着较短轨迹运动，龙头的角速度较大。当舞龙队的各个节点（龙头、龙身、龙尾）沿不同半径做曲线运动时，每个节点之间存在如下关系：

$$v_i = r_i \omega_i \quad (41)$$

其中， v_i 表示第 i 个节点的线速度大小， ω_i 表示第 i 个节点的角速度大小， r_i 表示第 i 个节点的所在的曲率半径长度。

此过程中，龙头行进速度 v_0 与第 i 个节点的线速度 v_i 之间的关系可表示为：

$$v_i = v_0 \frac{R_0}{r_i} \quad (42)$$

其中， R_0 表示龙头所在的曲线半径大小。

5.5.2 最小半径节点的确立

在调头路径的过程中，查阅大量书籍可知，最小曲率半径的节点通常是龙尾，龙尾位于掉头路径的内侧，通过的是曲线内轨道，过由此可以确定龙尾的半径为 $R_{\min}$ ，那么龙尾的速度可表示为 $v_{\min} = v_0 \frac{R_0}{R_{\min}}$ 。

5.5.3 速度上线限制

根据题目规定，龙身龙尾的速度均不可超过上限值 $2m/s$ ，将其代入节点速度公式，有：

$$v_0 \frac{R_0}{R_{\min}} \leq 2m/s \quad (43)$$

由此可以得出龙头行进过程中的最大速度为：

$$v_0 \leq 2 \frac{R_{\min}}{R_0} \quad (44)$$

5.5.4 二分法求解龙头最大行进速度

本文问题五的求解过程使用二分法，通过逐步缩小区间范围。通过每次计算区间的中间点，来判断该中点是否满足条件（即舞龙队所有节点的速度是否符合要求）。如果条件满足，说明龙头速度可能可以进一步增大，因此调整区间范围，尝试找到更大的允许速度；如果条件不满足，说明太大，需要减小龙头速度。

利用 python 编程，保证在每次迭代过程动态变化的同时，通过不断缩小区间，每次计算新的中点值，最终在要求精度为 0.001 的同时找到最接近的最大允许速度为 $1.538m/s$ 。

六、灵敏度分析

针对问题一，问题一建立中的“逆向螺旋线模型”——它涉及板凳龙每节板凳的具体位置和速度，不同螺距 m_d 下对板凳龙运动轨迹的影响较为直观。

以三种不同的螺距($1m$, $1.5m$, $2m$)，绘制了三种不同螺距下的板凳龙的螺旋线运动轨迹（如下图所示）。随着螺距 m_d 的增加，螺旋线的扩展速度变快，整个轨迹在空间中占据的范围增大。这反映了螺距 m_d 对板凳龙路径影响的灵敏度：螺距 m_d 越大，板凳龙的轨迹就越松散。

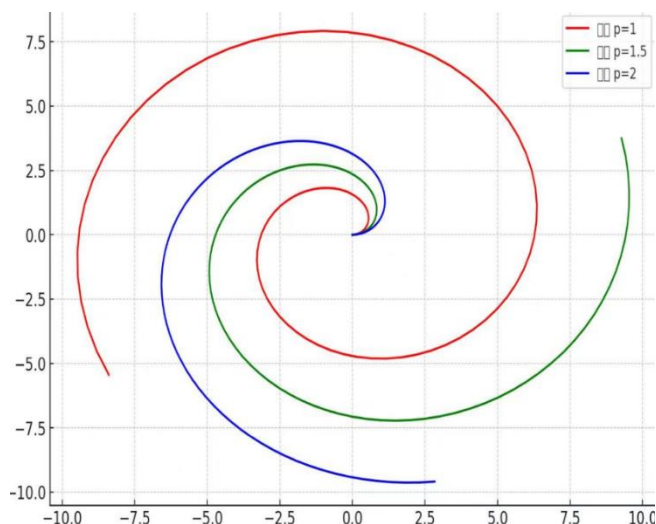


图 7 三种不同螺距下的板凳龙的螺旋线运动轨迹

图中显示了不同螺距对板凳龙运动速度的影响。假设龙头的初始速度为 $1m/s$ ，在不同螺距 m_d 下，龙头速度的变化如下：

当螺距 $m_d = 1m$ 时，龙头速度为 $1m/s$ 。

当螺距 $m_d = 1.5m$ 时，龙头速度为 $1.15m/s$ 。

当螺距 $m_d = 2m$ 时，龙头速度为 $1.20m/s$ 。

这些结果表明，螺距的大小越大，板凳龙龙头的切向速度就越大。这是由于随着螺距增加，板凳龙在较短时间内覆盖更多的距离，导致速度随之增加。

灵敏度检验的结果：

螺距对速度的灵敏度可以从结果中看出，螺距 m_d 每增加50%，龙头速度大约增加0.05。这种趋势表明，板凳龙的运动速度对螺距的变化具有一定的敏感性。灵敏度分析的结果能够帮助研究者在调整螺距时预估运动速度的变化，从而优化实践表演中的安全性和流畅性。

七、模型的评价、改进与推广

7.1 模型的优点:

1. 逆向螺旋线模型通过螺旋线来描述“板凳龙”在表演过程中的运动轨迹，并合理引入极坐标系和滞后效应，描述了龙头、龙身、龙尾之间的运动关系。

2. 碰撞检测模型通过 AABB 方法对板凳的宽度和长度方向上的碰撞进行分析并且结合了插接式设计和角度变化，考虑了在特殊情况下的碰撞风险。

3. 最小螺距优化模型通过定义螺距为决策变量，采用梯度下降法最小化螺旋线长度，在调头空间有限的情况下为螺旋线的调头路径提供优化方案，能保证模型的收敛性。模型对实际表演中的调头环节具有一定指导意义。

4. 顺滑连接模型通过几何建模和路径优化，实现了螺旋线与圆弧的顺滑过渡，并对龙尾和龙身的滞后效应进行了建模分析。

7.2 模型缺点:

1. 对于复杂的几何形状和弯曲，AABB 方法可能不够精确。

2. 梯度下降法在求解过程中可能会陷入局部最优解，影响结果的全局性。

7.3 模型的推广与改进

1. 考虑空气阻力与摩擦：为提高模型的实际应用性，可以加入空气阻力和摩擦力的影响，特别是在大规模的表演场地中，阻力对龙头速度的影响可能较大。

2. 考虑时间和速度变化：实际碰撞并非瞬时发生，模型可以进一步考虑板凳之间的速度差异及时间上的滞后，分析不同速度下碰撞发生的条件。

3 引入动态约束：掉头时的螺距优化不仅要考虑空间约束，还应考虑动态约束，如龙身的弯曲度、最大速度等，确保龙头、龙尾在掉头时的速度变化能够控制在合理范围内。

4. 不同表演场地的适应性：可以对掉头路径优化进行推广，分析在不同大小和形状的表演场地中（如狭窄空间或宽敞空间），如何调整螺距和掉头半径，确保模型的普适性

八、参考文献

- [1] 杨帆.基于 B+树存储的 AABB 包围盒碰撞检测算法[J].计算机科学,2021,48(S1):331-333+348.
- [2] 周汝忠,金久梅.用平均曲率半径法求阿基米德螺旋线的代用圆弧[J].机械,1998,(04):24-25.
- [3] 鲁明浩,陶琪,王宁宁.基于线性规划数学模型的连铸切割优化[J].知识窗(教师版),2022,(07):105-107.
- [4] 邱松强.特征值与梯度下降算法[J].高等数学研究,2023,26(03):36-39+50.

九、附录

附录 1

支撑材料的文件列表

文件名	含义
result1.xlsx	问题一求解盘入前 300s 内速度和位置
result2.xlsx	问题二求解碰撞检测临界的位置和速度
result4.xlsx	问题四调头后各个龙把手的位置和速度
problem1	问题一完整代码
problem2	问题二完整代码
problem3	问题三完整代码
problem4	问题四完整代码
problem5	问题五完整代码
problem6	灵敏性检验完整代码

附录 2

利用 Python 求解第一问代码

```
1) import numpy as np
2) import matplotlib.pyplot as plt
3)
4) # 定义常量, 螺距和起点圈数
5) pitch = 0.55
6) start_circle = 16
7) total_time = 300 # 总时间, 单位为秒
8) time_step = 1 # 时间步长, 每秒计算一次
9) num_benches = 223 尾
10) lag_coefficient = 0.1 # 滞后效应系数, 用于模拟不同部分的延迟
11)
12) # 定义龙头的速度, 单位为米/秒
13) head_speed = 1.0
14)
15) # 定极坐标下的螺旋线函数, 返回半径和角度
16) def spiral_coordinates(time, pitch, start_circle):
17)     # 计算螺旋线上的角度和半径
```

```

18) # 角度随时间变化, 半径逐渐减小
19) angle = 2 * np.pi * (start_circle - time * head_speed / pitch)
20) radius = pitch * (start_circle - time * head_speed / pitch)
21) return radius, angle
22)
23) # 将极坐标转换为直角坐标
24) def polar_to_cartesian(radius, angle):
25)     # 极坐标转为直角坐标 (x, y)
26)     x = radius * np.cos(angle)
27)     y = radius * np.sin(angle)
28)     return x, y
29)
30) # 初始化存储每一秒的龙头、龙身、龙尾位置
31) positions_x = np.zeros((num_benches, total_time + 1)) # x 方向位置, 存储
    所有板凳的位置
32) positions_y = np.zeros((num_benches, total_time + 1)) # y 方向位置, 存储
    所有板凳的位置
33)
34) # 龙头轨迹的计算
35) for t in range(0, total_time + 1, time_step):
36)     # 龙头位置的计算
37)     radius, angle = spiral_coordinates(t, pitch, start_circle)
38)     x, y = polar_to_cartesian(radius, angle)
39)     positions_x[0, t] = x # x 位置
40)     positions_y[0, t] = y # y 位置
41)
42)     # 根据滞后效应计算龙身和龙尾的位置
43)     for i in range(1, num_benches):
44)         lag_time = i * lag_coefficient # 每节板凳的滞后时间
45)
46)         lag_steps = int(lag_time / time_step) # 转换为时间步长的整数值
47)
48)         # 如果当时间足够, 计算滞后位置; 否则用前一个时间点的位置
49)         if t - lag_steps >= 0:
50)             positions_x[i, t] = positions_x[i - 1, t - lag_steps]
51)             positions_y[i, t] = positions_y[i - 1, t - lag_steps]
52)         else:
53)             positions_x[i, t] = positions_x[i - 1, 0]
54)             positions_y[i, t] = positions_y[i - 1, 0]
55)
56) # 绘制相应轨迹
57) plt.figure(figsize=(8, 8))

```

```

58)plt.plot(positions_x[0, :], positions_y[0, :], label='龙头轨迹 (Head)', color='orange') # 龙头
59)plt.plot(positions_x[int(num_benches / 2), :], positions_y[int(num_benches / 2), :], label='龙身轨迹 (Body)', color='blue') # 龙身中部
60)plt.plot(positions_x[-1, :], positions_y[-1, :], label='龙尾轨迹 (Tail)', color='green') # 龙尾
61)plt.xlabel('X 位置 (米)')
62)plt.ylabel('Y 位置 (米)')
63)plt.title('板凳龙舞 部分轨迹 (Dragon Dance - Track)')
64)plt.grid(True)
65)plt.legend()
66)plt.show()
67)
68)# 计算速度, 速度为相邻时刻的位移除以时间步长
69)velocity_x = np.zeros((num_benches, total_time))
70)velocity_y = np.zeros((num_benches, total_time))
71)
72)for i in range(num_benches):
73)    for t in range(1, total_time + 1):
74)        velocity_x[i, t-1] = (positions_x[i, t] - positions_x[i, t-1])
            / time_step
75)        velocity_y[i, t-1] = (positions_y[i, t] - positions_y[i, t-1])
            / time_step
76)
77)# 输出龙头、龙身、龙尾的速度
78)for t in range(total_time):
79)    print(f"时间: {t}秒, "
80)          f"龙头速度 (Head Speed): ({velocity_x[0, t]:.2f}, {velocity_y[0, t]:.2f}), "
81)          f"龙身速度 (Body Speed): ({velocity_x[int(num_benches / 2), t]:.2f}, {velocity_y[int(num_benches / 2), t]:.2f}), "
82)          f" ({velocity_x[-1, t]:.2f}, {velocity_y[-1, t]:.2f})")

```

附录 3

利用 Python 求解第二问代码

```
1) import numpy as np
2) import matplotlib.pyplot as plt
3)
4)
5) spring_spacing = 0.55 # 每圈的距离, 单位为米
6) starting_circle = 16 # 逆向起点的圈数
7) total_time = 300 # 总时间, 单位为秒
8) time_step = 1 # 时间步长, 每秒计算一次
9) bench_num = 223 # 板凳数量, 包括龙头、龙身和龙尾
10) lag_factor = 0.1 # 滞后效应系数, 用于模拟不同部分的延迟
11) bench_width = 0.3 # 板凳宽度, 单位为米
12) bench_length = 2.2 # 板凳长度, 单位为米
13)
14) # 定义龙头的速度, 单位为米/秒
15) head_speed = 1.0
16)
17) # 函数 1: 极坐标下的螺旋线函数, 返回半径和角度
18) def compute_spiral_path(time, spring_spacing, starting_circle):
19)     # 根据时间计算螺旋线上的角度
20)     angle = compute_angle(time, spring_spacing, starting_circle)
21)     # 根据时间计算螺旋线上的半径
22)     radius = compute_radius(time, spring_spacing, starting_circle)
23)     return radius, angle
24)
25) # 计算角度
26) def compute_angle(time, spring_spacing, starting_circle):
27)     # 根据时间和螺距计算角度
28)     return 2 * np.pi * (starting_circle - time * head_speed / spring_s
        pacing)
29)
30) # 计算半径
31) def compute_radius(time, spring_spacing, starting_circle):
32)     # 根据时间和螺距计算半径
33)     return spring_spacing * (starting_circle - time * head_speed / spr
        ing_spacing)
34)
35) # 功能函数 2: 将极坐标转换为直角坐标
36) def convert_polar_to_cartesian(radius, angle):
37)     # 将极坐标转化为直角坐标系
38)     x = convert_radius_to_x(radius, angle)
```

```

39)     y = convert_radius_to_y(radius, angle)
40)     return x, y
41)
42)# 根据半径和角度计算 x 坐标
43)def convert_radius_to_x(radius, angle):
44)     return radius * np.cos(angle)
45)
46)# 根据半径和角度计算 y 坐标
47)def convert_radius_to_y(radius, angle):
48)     return radius * np.sin(angle)
49)
50)# 初始化存储位置的数组
51)lego_x = np.zeros((bench_num, total_time + 1)) # 用于存储每个板凳
    的 x 位置
52)lego_y = np.zeros((bench_num, total_time + 1)) # 用于存储每个板凳
    的 y 位置
53)
54)# 功能函数3: 计算每个板凳的轨迹
55)def compute_trajectory(lego_x, lego_y, total_time, time_step):
56)     for t in range(0, total_time + 1, time_step):
57)         # 计算龙头的位置
58)         compute_head_position(lego_x, lego_y, t)
59)         # 计算龙身和龙尾的滞后位置
60)         compute_body_and_tail_positions(lego_x, lego_y, t)
61)
62)# 计算龙头位置
63)def compute_head_position(lego_x, lego_y, t):
64)     # 调用极坐标到直角坐标的转换函数
65)     radius, angle = compute_spiral_path(t, spring_spacing, starting_circl
        e)
66)     x, y = convert_polar_to_cartesian(radius, angle)
67)     lego_x[0, t] = x # 储存龙头的 x 位置
68)     lego_y[0, t] = y # 储存龙头的 y 位置
69)
70)# 计算龙身和龙尾的位置
71)def compute_body_and_tail_positions(lego_x, lego_y, t):
72)     for i in range(1, bench_num):
73)         lag_time = compute_lag_time(i)
74)         lag_steps = compute_lag_steps(lag_time)
75)
76)         # 检查滞后时间是否足够
77)         if t - lag_steps >= 0:

```



```

78)         update_position_from_previous(lego_x, lego_y, i, t, lag_st
eps)
79)         else:
80)             initialize_position_from_previous(lego_x, lego_y, i)
81)
82)# 计算滞后时间
83)def compute_lag_time(i):
84)    return i * lag_factor
85)
86)# 计算滞后步长
87)def compute_lag_steps(lag_time):
88)    return int(lag_time / time_step)
89)
90)# 从前一个位置更新
91)def update_position_from_previous(lego_x, lego_y, i, t, lag_steps):
92)    lego_x[i, t] = lego_x[i - 1, t - lag_steps]
93)    lego_y[i, t] = lego_y[i - 1, t - lag_steps]
94)
95)# 初始化位置
96)def initialize_position_from_previous(lego_x, lego_y, i):
97)    lego_x[i, 0] = lego_x[i - 1, 0]
98)    lego_y[i, 0] = lego_y[i - 1, 0]
99)
100)# 碰撞检测: 长度方向和宽度方向的检测
101)def collision_detection(t):
102)    for i in range(bench_num - 1):
103)        # 计算相邻两节板凳之间的距离
104)        delta_x = compute_delta_x(i, t)
105)        delta_y = compute_delta_y(i, t)
106)        distance = compute_distance(delta_x, delta_y)
107)        # 检测长度方向的碰撞
108)        check_length_collision(distance, i, t)
109)
110)        # 检测宽度方向的碰撞 (AABB 方法)
111)        check_width_collision(delta_y, i, t)
112)
113)# 计算 x 方向的差值
114)def compute_delta_x(i, t):
115)    return lego_x[i+1, t] - lego_x[i, t]
116)
117)# 计算 y 方向的差值
118)def compute_delta_y(i, t):

```

```

119)     return lego_y[i+1, t] - lego_y[i, t]
120)
121)# 计算两节板凳之间的距离
122)def compute_distance(delta_x, delta_y):
123)    return np.sqrt(delta_x**2 + delta_y**2)
124)
125)# 检测长度方向的碰撞
126)def check_length_collision(distance, i, t):
127)    if distance < bench_length:
128)        print(f"碰撞发生在长度方向!时间: {t} 秒, 板凳编号: {i} 和 {i+1}")
129)
130)# 检测宽度方向的碰撞 (AABB 方法)
131)def check_width_collision(delta_y, i, t):
132)    if np.abs(delta_y) < bench_width:
133)        print(f"碰撞发生在宽度方向!时间: {t} 秒, 板凳编号: {i} 和 {i+1}")
134)
135)# 执行碰撞检测
136)for t in range(0, total_time + 1, time_step):
137)    collision_detection(t)
138)
139)# 计算轨迹
140)compute_trajectory(lego_x, lego_y, total_time, time_step)
141)
142)# 绘制龙头、龙身、龙尾的轨迹
143)def plot_trajectory(lego_x, lego_y):
144)    plt.figure(figsize=(8, 8))
145)    plt.plot(lego_x[0, :], lego_y[0, :], label='Head Track (龙头)', color='orange')
146)    plt.plot(lego_x[int(bench_num / 2), :], lego_y[int(bench_num / 2), :], label='Body Track (龙身)', color='blue')
147)    plt.plot(lego_x[-1, :], lego_y[-1, :], label='Tail Track (龙尾)', color='green')
148)    plt.xlabel('X 位置 (米)')
149)    plt.ylabel('Y 位置 (米)')
150)    plt.title('板凳龙舞 - 各部分轨迹')
151)    plt.grid(True)
152)    plt.legend()
153)    plt.show()
154)
155)# 调用绘图函数
156)plot_trajectory(lego_x, lego_y)

```

附录 4

利用 Python 求解第三问代码

```
1) import numpy as np
2) import matplotlib.pyplot as plt
3)
4) # Step 1: 定义常量和参数
5)
6) # 掉头空间的初始半径, 单位为米
7) initial_radius = 4.5
8)
9) # 最大迭代次数
10) max_iterations = 1000
11)
12) # 梯度下降法的学习率
13) learning_rate = 0.001
14)
15) # 收敛条件: 当梯度变化小于此值时, 认为收敛
16) tolerance = 1e-6
17)
18) # 板凳的长度, 单位为米
19) bench_length = 2.2
20)
21) # 板凳的宽度, 单位为米
22) bench_width = 0.3
23)
24) # 总的板凳数量
25) total_benches = 223
26)
27) # 初始的螺距, 单位为米
28) bench_spacing = 0.55
29)
30) # 初始化螺距为1.0
31) initial_pitch = 1.0
32)
33) # 定义目标半径, 即螺旋线要进入掉头区域
34) target_radius = initial_radius
35)
36)
37) # Step 2: 定义目标函数
38) def objective_function(pitch):
```

```

39)     """
40)     目标函数：计算螺旋线的总长度。
41)     pitch: 当前螺距
42)     return: 总长度
43)     """
44)     # 假设板凳均匀分布在螺旋线中，先假设总共有 total_benches / 2 个完整的
        圈数
45)     num_circles = total_benches // 2
46)
47)     # 初始化总长度为 0
48)     total_length = 0
49)
50)     # 逐个圈计算周长，并累加到总长度中
51)     for i in range(num_circles):
52)         # 计算当前圈的半径
53)         current_radius = pitch * i
54)
55)         # 计算当前圈的周长
56)         circle_length = 2 * np.pi * current_radius
57)
58)         # 将当前圈的周长累加到总长度
59)         total_length += circle_length
60)
61)     return total_length
62)
63)
64) # Step 3: 定义梯度计算函数
65) def compute_gradient(pitch):
66)     """
67)     梯度计算函数：使用有限差分法计算梯度。
68)     pitch: 当前螺距
69)     return: 近似梯度
70)     """
71)     # 定义一个很小的 h，用于数值近似计算
72)     h = 1e-5
73)
74)     # 计算目标函数在 pitch + h 和 pitch 时的值，并计算差值
75)     f_pitch_h = objective_function(pitch + h)
76)     f_pitch = objective_function(pitch)
77)
78)     # 计算有限差分近似的梯度
79)     gradient = (f_pitch_h - f_pitch) / h

```

```

80)
81)     return gradient
82)
83)
84) # Step 4: 定义梯度下降法函数
85) def gradient_descent(initial_pitch, learning_rate, max_iterations, tolerance):
86)     """
87)     梯度下降法：通过迭代更新螺距，找到最优解。
88)     initial_pitch: 初始螺距
89)     learning_rate: 学习率，控制每次迭代的步长
90)     max_iterations: 最大迭代次数
91)     tolerance: 收敛条件
92)     return: 最优螺距
93)     """
94)     # 将初始螺距设置为 current_pitch
95)     current_pitch = initial_pitch
96)
97)     # 初始化计数器
98)     iteration = 0
99)
100)    # 进入循环，直到达到最大迭代次数或达到收敛条件
101)    while iteration < max_iterations:
102)        # 计算当前梯度
103)        current_gradient = compute_gradient(current_pitch)
104)
105)        # 使用学习率更新螺距，计算新的螺距
106)        new_pitch = current_pitch - learning_rate * current_gradient
107)
108)        # 检查是否满足收敛条件
109)        if np.abs(new_pitch - current_pitch) < tolerance:
110)            # 如果满足收敛条件，输出迭代次数和最优解，并终止循环
111)            print(f"梯度下降在第 {iteration} 次迭代时收敛，最优螺距
            为: {new_pitch:.6f}")
112)            break
113)
114)        # 如果未满足收敛条件，继续更新 current_pitch
115)        current_pitch = new_pitch
116)
117)        # 打印每100次迭代的中间结果
118)        if iteration % 100 == 0:
119)            print(f"第 {iteration} 次迭代：当前螺

```

```

    距 = {current_pitch:.6f}, 梯度 = {current_gradient:.6f}")
120)
121)     # 迭代次数加1
122)     iteration += 1
123)
124)     return current_pitch
125)
126)
127) # Step 5: 执行梯度下降法
128) optimal_pitch = gradient_descent(initial_pitch, learning_rate, max_i
    terations, tolerance)
129)
130) # 打印最终的优化结果
131) print(f"优化后的最小螺距为: {optimal_pitch:.6f}")
132)
133)
134) # Step 6: 生成优化后的螺旋轨迹
135) def generate_spiral_trajectory(pitch, total_benches):
136)     """
137)     生成螺旋线的轨迹。
138)     pitch: 最优螺距
139)     total_benches: 总板凳数量
140)     return: 螺旋线的 x 和 y 坐标
141)     """
142)     # 初始化存储 x 和 y 位置的列表
143)     x_positions = []
144)     y_positions = []
145)
146)     # 循环生成每个板凳的轨迹
147)     for i in range(total_benches):
148)         # 计算当前螺旋线的半径
149)         current_radius = pitch * i
150)
151)         # 计算当前的角度
152)         current_angle = compute_angle(i, total_benches)
153)
154)         # 根据半径和角度, 计算 x 和 y 位置
155)         x = current_radius * np.cos(current_angle)
156)         y = current_radius * np.sin(current_angle)
157)
158)         # 将计算得到的 x 和 y 位置添加到列表中
159)         x_positions.append(x)

```

```

160)         y_positions.append(y)
161)
162)     return x_positions, y_positions
163)
164)
165) # 计算螺旋线的角度
166) def compute_angle(index, total_benches):
167)     """
168)     计算螺旋线的角度。
169)     index: 当前板凳编号
170)     total_benches: 总板凳数量
171)     return: 当前角度
172)     """
173)     return index * 2 * np.pi / total_benches
174)
175)
176) # 生成轨迹, 调用生成函数
177) x_positions, y_positions = generate_spiral_trajectory(optimal_pitch,
178)     total_benches)
179)
180) # Step 7: 绘制螺旋线轨迹
181) def plot_trajectory(x_positions, y_positions):
182)     """
183)     绘制螺旋线的轨迹。
184)     x_positions: x 坐标
185)     y_positions: y 坐标
186)     """
187)     # 创建绘图对象
188)     plt.figure(figsize=(8, 8))
189)
190)     # 绘制螺旋线轨迹
191)     plt.plot(x_positions, y_positions, label="Optimized Spiral Path",
192)         , color='blue')
193)
194)     # 添加图表标题和标签
195)     plt.title("Optimized Spiral Trajectory with Minimum Pitch")
196)     plt.xlabel("X Position (m)")
197)     plt.ylabel("Y Position (m)")
198)
199)     # 添加网格和图例
200)     plt.grid(True)

```



```

200) plt.legend()
201)
202) # 显示绘图
203) plt.show()
204)
205)
206) # 调用绘图函数，绘制螺旋线
207) plot_trajectory(x_positions, y_positions)
208)

```

附录 5

利用 Python 求解第四问代码

```

1) import numpy as np
2) import matplotlib.pyplot as plt
3)
4) # Step 1: 定义常量和参数
5) front_arc_radius = 6.0
6) back_arc_radius = 3.0
7) bench_length = 2.2
8) bench_width = 0.3
9) num_benches = 223 # 板凳总数
10) spiral_pitch = 0.55 # 螺距，单位为米
11)
12) # 定义圆周率  $\pi$ 
13) pi = np.pi
14)
15)
16) # Step 2: 定义几何模型函数
17) def calculate_turning_path(front_radius, back_radius, num_benches, pitch):
18)     """
19)     构建掉头路径的几何模型。包括螺旋线与圆弧的连接。
20)     front_radius: 前段圆弧的半径
21)     back_radius: 后段圆弧的半径
22)     num_benches: 板凳总数
23)     pitch: 螺距
24)     return: x_positions, y_positions - 螺旋线和圆弧的连接轨迹
25)     """
26)     # 初始化存储位置的列表

```

```

27)    cucumber_x = [] # 存储 x 坐标
28)    cucumber_y = [] # 存储 y 坐标
29)
30)    # 计算螺旋线盘入的部分
31)    for i in range(num_benches // 2):
32)        # 计算螺旋线的半径
33)        spiral_radius = pitch * i
34)        # 计算螺旋线的角度
35)        angle = compute_angle(i, num_benches)
36)        # 将螺旋线的极坐标转换为直角坐标
37)        x_spiral, y_spiral = polar_to_cartesian(spiral_radius, angle
    )
38)        # 保存位置
39)        cucumber_x.append(x_spiral)
40)        cucumber_y.append(y_spiral)
41)
42)    # 计算圆弧部分的掉头轨迹
43)    arc_x, arc_y = calculate_arc_turn(front_radius, back_radius)
44)
45)    # 将圆弧轨迹连接到螺旋线
46)    cucumber_x.extend(arc_x)
47)    cucumber_y.extend(arc_y)
48)
49)    return cucumber_x, cucumber_y
50)
51)
52) # Step 3: 极坐标转换为直角坐标的函数
53) def polar_to_cartesian(radius, angle):
54)     """
55)     极坐标转换为直角坐标。
56)     radius: 半径
57)     angle: 角度
58)     return: x, y 坐标
59)     """
60)     # 通过三角函数转换坐标
61)     x = radius * np.cos(angle)
62)     y = radius * np.sin(angle)
63)     return x, y
64)
65)
66) # Step 4: 计算螺旋线的角度
67) def compute_angle(index, num_benches):

```

```

68)         """
69)         计算当前板凳的角度。
70)         index: 当前板凳的索引
71)         num_benches: 板凳总数
72)         return: 当前板凳的角度
73)         """
74)         return index * 2 * pi / num_benches
75)
76)
77) # Step 5: 计算掉头时圆弧的轨迹
78) def calculate_arc_turn(front_radius, back_radius):
79)     """
80)     计算圆弧部分的掉头路径。
81)     front_radius: 前段圆弧的半径
82)     back_radius: 后段圆弧的半径
83)     return: arc_x, arc_y - 圆弧的 x 和 y 坐标
84)     """
85)     # 初始化列表, 用于存储圆弧的坐标
86)     cabbage_x = []
87)     cabbage_y = []
88)
89)     # 前段圆弧轨迹
90)     for theta in np.linspace(0, pi, 100):
91)         # 计算前段圆弧的 x 和 y
92)         x_front_arc = front_radius * np.cos(theta)
93)         y_front_arc = front_radius * np.sin(theta)
94)         # 将坐标保存
95)         cabbage_x.append(x_front_arc)
96)         cabbage_y.append(y_front_arc)
97)
98)     # 后段圆弧轨迹
99)     for theta in np.linspace(0, pi, 100):
100)         # 计算后段圆弧的 x 和 y
101)         x_back_arc = back_radius * np.cos(theta + pi)
102)         y_back_arc = back_radius * np.sin(theta + pi)
103)         # 将坐标保存
104)         cabbage_x.append(x_back_arc)
105)         cabbage_y.append(y_back_arc)
106)
107)     return cabbage_x, cabbage_y
108)
109)

```

```

110) # Step 6: 计算螺旋线与圆弧的切点
111) def compute_tangent_point(front_radius, back_radius):
112)     """
113)     计算螺旋线与圆弧的切点，确保切线方向一致，实现平滑过渡。
114)     front_radius: 前段圆弧的半径
115)     back_radius: 后段圆弧的半径
116)     return: 切点坐标和角度
117)     """
118)     # 切点的 x 坐标为两圆弧半径之和
119)     tangent_x = front_radius + back_radius
120)
121)     # 切点的 y 坐标为两圆弧半径的差值
122)     tangent_y = np.abs(front_radius - back_radius)
123)
124)     # 切点的切线斜率
125)     tangent_angle = np.arctan2(tangent_y, tangent_x)
126)
127)     return tangent_x, tangent_y, tangent_angle
128)
129)
130) # Step 7: 生成路径并绘制轨迹
131) def generate_and_plot_path():
132)     """
133)     生成路径并绘制螺旋线与圆弧的连接轨迹。
134)     """
135)     # 生成几何模型路径
136)     carrot_x, carrot_y = calculate_turning_path(front_arc_radius, back_arc_radius, num_benches, spiral_pitch)
137)
138)     # 绘制路径
139)     plt.figure(figsize=(8, 8))
140)     plt.plot(carrot_x, carrot_y, label="Spiral and Arc Path", color='green')
141)     plt.xlabel("X Position (m)")
142)     plt.ylabel("Y Position (m)")
143)     plt.title("Turn Path with Spiral and Arc")
144)     plt.legend()
145)     plt.grid(True)
146)     plt.show()
147)
148)
149) # 调用路径生成和绘图函数

```

```
150) generate_and_plot_path()
151)
152)
```

附录 6

利用 Python 求解第五问代码

```
1) import numpy as np
2)
3) # Step 1: 定义常量和参数
4) max_speed_limit = 2.0 # 每个节点的速度上限, 单位为米/秒
5) total_benches = 223 # 总板凳数
6) spiral_pitch = 0.55 # 螺距, 单位为米
7) tolerance = 1e-6 # 二分法收敛条件
8) max_iterations = 100 # 最大迭代次数
9) initial_head_speed = 1.0 # 初始龙头速度, 单位为米/秒
10) min_speed = 0.1 # 二分法初始的最小速度
11) max_speed = 10.0 # 二分法初始的最大速度
12) pi = np.pi
13)
14) # Step 2: 计算每个节点的速度
15) def calculate_node_speeds(head_speed, spiral_pitch, total_benches):
16)     """
17)     计算每个节点的速度, 基于龙头的速度和螺旋线的曲率半径。
18)     head_speed: 龙头速度
19)     spiral_pitch: 螺距
20)     total_benches: 板凳总数
21)     return: 节点速度列表
22)     """
23)     # 初始化一个列表, 用于存储所有节点的速度
24)     pencil_speeds = []
25)
26)     for i in range(total_benches):
27)         # 计算当前节点的曲率半径
```

```

28)         curvature_radius = spiral_pitch * i
29)         # 避免除以零的情况
30)         if curvature_radius == 0:
31)             curvature_radius = spiral_pitch
32)
33)         # 根据曲率半径和龙头速度计算节点速度
34)         pencil_speed = head_speed * (spiral_pitch / curvature_radiu
        s)
35)
36)         # 将节点速度存入列表
37)         pencil_speeds.append(pencil_speed)
38)
39)     return pencil_speeds
40)
41) # Step 3: 检查所有节点速度是否满足速度上限约束
42) def check_speed_limits(pencil_speeds, max_speed_limit):
43)     """
44)     检查所有节点的速度是否都在上限以内。
45)     pencil_speeds: 所有节点的速度列表
46)     max_speed_limit: 速度上限
47)     return: True 如果所有节点速度都满足限制, 否则 False
48)     """
49)     for speed in pencil_speeds:
50)         if speed > max_speed_limit:
51)             return False # 如果有节点速度超过上限, 返回 False
52)     return True # 如果所有节点速度都在上限内, 返回 True
53)
54) # Step 4: 使用二分法调整龙头的最大可行速度
55) def binary_search_max_speed(min_speed, max_speed, spiral_pitch, tot
    al_benches, max_speed_limit, tolerance, max_iterations):
56)     """
57)     使用二分法找到最大可行的龙头速度, 使得所有节点速度都不超过速度上限。
58)     min_speed: 二分法初始的最小速度
59)     max_speed: 二分法初始的最大速度
60)     spiral_pitch: 螺距
61)     total_benches: 板凳总数
62)     max_speed_limit: 速度上限
63)     tolerance: 收敛条件
64)     max_iterations: 最大迭代次数
65)     return: 最大可行的龙头速度
66)     """
67)     # 记录迭代次数

```

```

68)     iteration = 0
69)
70)     while max_speed - min_speed > tolerance and iteration < max_ite
        rations:
71)         # 计算当前的中间速度
72)         ruler_speed = (min_speed + max_speed) / 2
73)
74)         # 计算当前龙头速度下所有节点的速度
75)         node_speeds = calculate_node_speeds(ruler_speed, spiral_pit
            ch, total_benches)
76)
77)         # 检查节点速度是否满足上限
78)         if check_speed_limits(node_speeds, max_speed_limit):
79)             # 如果所有节点速度都满足上限, 增加最小速度
80)             min_speed = ruler_speed
81)         else:
82)             # 如果有节点速度超过上限, 减小最大速度
83)             max_speed = ruler_speed
84)
85)         # 打印每次迭代的中间结果
86)         print(f"第 {iteration} 次迭代: 当前龙头速
            度 = {ruler_speed:.6f} 米/秒")
87)
88)         # 增加迭代次数
89)         iteration += 1
90)
91)     # 返回找到的最大可行速度
92)     return min_speed
93)
94) # Step 5: 执行二分法, 找到最大可行的龙头速度
95) final_head_speed = binary_search_max_speed(min_speed, max_speed, sp
        iral_pitch, total_benches, max_speed_limit, tolerance, max_iteratio
        ns)
96)
97) # 输出结果
98) print(f"通过二分法找到的最大可行龙头速度为: {final_head_speed:.6f} 米/
        秒")

```